Machine Learning Lecture RAG Assistant

Project Report — Retrieval-Augmented Generation with LangChain, LangGraph, Ollama, ChromaDB, and FastAPI

1. Objective

The objective is to build a question-answering application over a collection of Machine Learning lecture PDFs. The system retrieves relevant passages from the supplied documents and uses a local language model to generate answers grounded in the retrieved context. When the context is insufficient, the prompt instructs the model to respond: *“I don't know based on the provided documents.”*

2. Dataset and Implementation

The document collection contains three PDFs: **ML_lecture1.pdf**, **ML2_Lecture2.pdf**, and **ML2_Lecture3.pdf**. PyPDFLoader extracted 103 pages. RecursiveCharacterTextSplitter was configured with a chunk size of 1,000 characters and an overlap of 200 characters. The resulting 104 chunks were embedded using Ollama **nomic-embed-text** and stored persistently in ChromaDB under the **ml_lectures** collection.

At query time, Chroma similarity search retrieves the top 3 chunks. The documented chat model is Ollama **llama3.2:3b** (use the exact tag configured in the project). The model generates an answer using a prompt restricted to the retrieved context. Source file names and page metadata are returned with the answer.

3. LangGraph Workflow

START → retrieve → generate → END

The graph uses a typed state containing the question, retrieved documents, and generated answer. The retrieve node calls the Chroma retriever; the generate node builds the context, invokes the chat model, and returns the answer. The workflow returns the answer together with source metadata.

4. Backend and API

FastAPI exposes health checks and the RAG functionality. The API routes documented in the project are:

Method	Route	Purpose
GET	/	Root/API check
GET	/health	Health status
POST	/ask	Ask a question; returns an answer and source metadata
POST	/upload	Upload a PDF into the documents folder
POST	/embed	Rebuild the vector database and return the stored chunk count

POST /upload: Accepts a file using multipart/form-data with field name **file**. The implementation checks that a filename exists, accepts only the .pdf extension, rejects empty files, limits uploads to 20 MB, and checks that the content begins with the PDF signature (%PDF). It creates the documents directory if needed, generates a unique filename, saves the PDF, and returns a success message and saved filename.

POST /embed: Calls **create_vectorstore()** to rebuild the vector database, counts the IDs stored in the vector store, and returns a success message with **chunks_stored**. This route can be used after uploading new documents so that they are embedded into the collection.

The interactive API documentation is available at **http://127.0.0.1:8000/docs** while the server is running.

5. Evaluation

Test	Observed result	Assessment
PDF ingestion	103 pages loaded from 3 PDFs	Passed
Chunking	104 chunks created	Passed
Vector storage	104 chunks stored in Chroma	Passed

Test	Observed result	Assessment
Retrieval	Top 3 returned; specific questions were more relevant than broad questions	Functional; tune further
In-scope generation	NLP question returned an answer; sources: ML_lecture1.pdf pages 8, 7, 11	Passed initial test
Out-of-scope generation	Returned “I don't know based on the provided documents.”	Passed
LangGraph	retrieve → generate returned answer and sources	Passed
FastAPI	/ask tested; malformed JSON returned HTTP 422	Validation observed

The /upload and /embed endpoints are included in the supplied FastAPI implementation. Their behavior is described in this report; no separate test result for these two routes was included in the original evaluation notes.

6. Analysis and Limitations

The implementation demonstrates the local RAG flow: PDF ingestion, chunking, embedding, persistent vector storage, similarity retrieval, grounded generation, source metadata, and LangGraph orchestration. Initial tests showed that specific questions about lecture topics retrieved more useful passages than a broad question such as “What is Machine Learning?”. Retrieval quality therefore depends on query specificity. The current retriever uses top-k = 3; chunk size, overlap, and k can be tuned in future work.

An NVIDIA CUDA out-of-memory issue was encountered. Running Ollama with CPU execution succeeded, although CPU inference may be slower. The evaluation is an initial functional check rather than a comprehensive benchmark: no numeric retrieval metrics or answer-quality scores were calculated. Returned source pages should be checked against the original PDFs when assessing factual correctness.

7. How to Run

```
pip install -r requirements.txt
ollama pull nomic-embed-text
ollama pull llama3.2:3b  # or the exact chat model configured in the project
python -m app.embeddings
uvicorn app.main:app --reload
```

Then open Swagger UI at **http://127.0.0.1:8000/docs**. Keep local secrets in .env and do not commit them. The .env.example file should contain safe example values only.